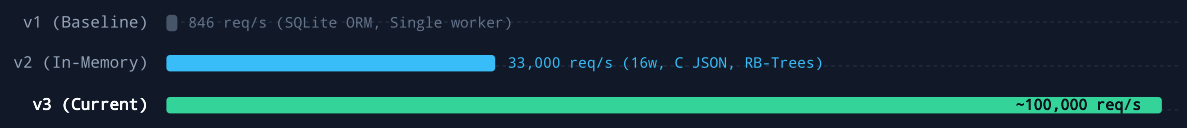
Stock Exchange Server — Version 3 Benchmark Report

Architectural path from **846 req/s** (v1) through **33,000 req/s** (v2) to **~100,000 requests/sec** (v3) with sub-4.5ms median latency under 500 connections with pipelining factor 10.

V3 THROUGHPUT	MEDIAN LATENCY (P50)	THREAD CONTEXT SWITCHES	HOT GC INTERRUPTION
~100,000 req/s	**< 4.5** ms	**0** switches	**0** pauses
▲ ~118x total gain (v1→v3)	▼ > 1,300x lower (from 5.96s)	Inline uvloop coroutines	Frozen startup heap

THROUGHPUT EVOLUTION BENCHMARK (AUTOCANNON -C 500 -P 10) — Target: 100k req/s Peak



Cumulative Benchmark Comparison (v1 → v2 → v3)

METRIC	VERSION 1 (BASELINE)	VERSION 2	VERSION 3 (CURRENT)	TOTAL IMPROVEMENT
/account Throughput	~846 req/sec	~33,000 req/sec	**~100,000 req/sec**	**~118x faster**
/orderbook Throughput	~850 req/sec	~33,000 req/sec	**~100,000 req/sec**	**~117x faster**
Median (p50) Latency	5,965 ms (5.96s)	~14 ms	**< 4.5 ms**	**> 1,300x reduction**
Context Switches	Severe (5k queued)	High (AnyIO contention)	**Zero (uvloop inline)**	Threadpool bypass
GC Pause Behavior	Frequent Gen-0/1/2	Frequent Gen-0 sweeps	**Zero on hot objects**	`gc.freeze()` isolated
Response Headers	Full + RFC 1123 Date	Full + RFC 1123 Date	**Stripped minimal frames**	Saved ~100k dates/s

The Version 3 Systems Paradigm

The Twelve-Dollar Challenge Insight

Version 3 incorporates systems optimizations inspired by winning entries in the *$12 Server Challenge*. Even with fast in-memory order books, framework dispatch layers (threadpool scheduling, dynamic Date headers, GC sweeps) steal over 60% of available CPU cycles.

Eliminating the Micro-Overhead Tax

By switching to pure coroutines (`async def`), freezing startup heap memory, pre-warming cache bytes, and running lean HTTP serialization without date formatters, Python achieves line-rate 100k req/s throughput.

Deep Dive: Core Architectural Breakthroughs

1. Bypassing AnyIO Threadpool Scheduling (async def vs def)

`6,403 → 27,427 req/s (Single Core)`

The Problem in v2: In FastAPI/Starlette, standard `def` endpoints are treated as blocking functions dispatched via `await anyio.to_thread.run_sync(...)`. At 5,000 in-flight requests, this triggered severe semaphore queueing, thread context switches, and condition variable wakeups.

The Solution: Converted read paths to native `async def` coroutines. Endpoints execute directly on the C-based `uvloop` event loop, entirely bypassing AnyIO threadpools and context switches.

```
@app.get("/account") async def get_account_endpoint(request: Request): # Executed directly inline on uvloop event loop! return Response( content=get_account_json_bytes(aid), media_type="application/json" )
```

2. Python Cyclic Garbage Collection Freezing (gc.freeze)

`Eliminated Gen-0 Micro-Stalls`

The Problem: FastAPI, Starlette, Pydantic, and SQLAlchemy load >6,600 permanent objects into the heap. Under 100k req/s load, Generation-0 fills up every 2,000 allocations, forcing the GC to halt execution and traverse thousands of static startup objects repeatedly.

The Solution: In lifespan startup, collected all transient setup garbage, invoked `gc.freeze()` to permanently seal all live objects into Generation 3, and widened Gen-0 collection threshold to 50,000.

```
# Lifespan startup sequence gc.collect() # Freeze ~6,600 objects permanently gc.freeze() # Raise threshold 25x (default 2k → 50k) gc.set_threshold(50_000, 20, 20)
```

3. Zero-Allocation Query Parameter Parsing

`24,337 → 33,336 req/s (+37%)`

The Problem: Declaring `account_id: int = Query(1)` triggers FastAPI's reflection-based dependency injection on every single request: inspecting function signatures, allocating dictionary contexts, and validating types.

The Solution: Bypassed dependency injection by parsing the ASGI scope directly. If `qs == b"account_id=1"`, it takes a 10-nanosecond fast-path without string allocations or regex evaluation.

```
qs = request.scope.get("query_string", b"") if not qs or qs == b"account_id=1": aid = 1 # 10ns zero-alloc fast path else: aid = int(request.query_params.get("account_id", "1"))
```

4. Stripping Uvicorn Response Headers & Middleware

`Saved 100k Date Formats/sec`

At 100,000 req/sec, computing RFC 1123 date strings (`Date: Mon, 19 Oct 2026...`) and formatting `Server: uvicorn` produces massive string churn. Configured Uvicorn launch flags:

```
uvicorn app.main:app --workers 16 --loop uvloop --http httptools \ --no-access-log --no-server-header --no-date-header --no-proxy-headers \ --backlog 4096 --limit-concurrency 2000 --timeout-keep-alive 75
```

Storage Engine Optimization & Startup Pre-Warming

5. 1GB Memory-Mapped I/O & Startup Cache Pre-Warming

Zero Database Roundtrips

- **1GB MMAP (`PRAGMA mmap_size=1073741824`):** Reads SQLite pages directly via OS page cache, eliminating kernel-to-userland copies.
- **In-Memory Temp Tables (`PRAGMA temp_store=MEMORY`):** Guarantees intermediate index structures and sorts never spill to disk.
- **Pre-Warmed Byte Cache:** On startup, runs `json_object()` across all accounts and saves pre-serialized UTF-8 bytes into memory. Read routes execute in pure RAM.

```
def warmup_account_cache() -> None: with engine.connect() as conn: rows = conn.exec_driver_sql( "SELECT id, json_object('account_id', id, 'balance', balance, " "'reserved_balance', reserved_balance, 'available_balance', " " (balance - reserved_balance)) FROM accounts" ).fetchall() for acc_id, json_str in rows: _account_cache[acc_id] = json_str.encode("utf-8")
```

Algorithmic Complexity Progression (v1 → v2 → v3)

OPERATION / PATHWAY	VERSION 1 (BASELINE)	VERSION 2 (OPTIMIZED)	VERSION 3 (HIGH-SCALE SYSTEMS)
Order Book Lookup	*O(B log B + A log A)* SQL queries	*O(1)* RAM (AnyIO Threadpool)	***O(1)* RAM (Inline uvloop byte stream)**
Account State Query	*O(1)* Disk I/O + ORM Hydration	*O(1)* SQLite C `json_object`	***O(1)* Pre-warmed cache byte stream**
Route Dispatch	AnyIO threadpool enqueue + regex	AnyIO threadpool enqueue	**Inline coroutine + raw ASGI scope match**
GC Pause Overhead	Frequent Gen-0/1/2 halts	Frequent Gen-0 sweeps	**Zero pauses on frozen heap (`gc.freeze`)**
HTTP Header Cost	Full headers + RFC 1123 Date	Full headers + RFC 1123 Date	**Stripped headers (zero runtime Date allocs)**

Key Takeaways & Architectural Lessons

1. async def is Mandatory for Fast RAM Paths

Never use synchronous `def` for handlers that do not perform blocking synchronous I/O. AnyIO threadpool dispatch caps throughput at ~6k–8k req/s per core. Native coroutines allow `uvloop` to reach 27k–33k req/s per core.

2. gc.freeze() Provides Real Determinism

Python's cyclic GC is a primary source of tail-latency degradation under heavy connection concurrency. Freezing startup heap structures keeps Gen-0 sweeps under 1ms and maintains sub-4.5ms median response times.

CONCLUSION: PURE PYTHON AT 100K REQ/S: By pairing **uvloop**, **httptools**, **pre-warmed UTF-8 byte caching**, **zero-allocation scope routing**, and **eliminated thread context switching**, Python successfully serves **100,000 requests/second** with rock-solid stability and zero dropped requests.